kernscope: A Unified C++ GPU-Kernel Tooling Suite

Static Analysis, Autotuning, Verification, Capture/Replay, LLVM Passes, and Profiling in One Repository

Research Report

August 15, 2026

Abstract

GPU kernel development today is fragmented across vendor profilers, simulators, ad-hoc autotuners, and one-off static analyses that rarely share infrastructure or validate each other's outputs. This report describes **kernscope**, a unified C++ GPU-kernel tooling suite that integrates *six* command-line tools — a PTX/AMDGPU static analyzer, a CUDA/HIP autotuning kernel generator, a static GPU kernel verifier, a kernel capture-and-replay system, an out-of-tree LLVM-21 pass plugin, and a tiny CUDA/ROCm profiler — on top of one shared library stack (`libs/`). The six tools form one closed loop: a kernel enters as source or a running binary and leaves with provenance-labeled, cross-validated evidence (profile → capture → analyze → transform/generate → verify → benchmark). All engineering is grounded in the project's own research briefs, which survey GPUVerify, PUG, ESBMC-GPU, Kerncap, HiRace, HGRD, Faial, IREE, Triton, FlashAttention, and the eBPF verifier as prior art. Every measured number in this report was collected on a local AMD Radeon RX 7800 XT (gfx1101) — measured peaks of 591 GB/s DRAM read bandwidth, 16.32 TFLOPS fp32, and 9.7 TB/s LDS — under a strict honesty regime: reports carry `provenance.mode` = *measured* | *projected*, NVIDIA runtime paths are compile-only and owner-gated rentals, and *no* NVIDIA runtime numbers are claimed.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Contributions . . . 3
- 1.3 Reading guide . . . 3

2 Background and Related Work **4**
- 2.1 Roofline and the occupancy model (foundational) . . . 4
- 2.2 Static analysis of GPU assembly (C1) . . . 4
- 2.3 Autotuning and tensor-program compilation (C2) . . . 4
- 2.4 GPU kernel verification (C3) . . . 4
- 2.5 Capture / replay (C4) . . . 5
- 2.6 LLVM passes (C5) . . . 5
- 2.7 Profiling (C6) . . . 5

3 System Architecture **5**
- 3.1 The closed tool loop . . . 5
- 3.2 Library dependency graph . . . 6
- 3.3 Dual-path hardware strategy . . . 6

4 The Six Tools **6**
- 4.1 C1 — `kernscope-analyze`: static analyzer . . . 6
- 4.2 C2 — `kernscope-tune`: autotuning kernel generator . . . 6

4.3 C3 — `kernscope-verify`: static verifier . . . 7
4.4 C4 — `kernscope-capture`: capture and replay . . . 7
4.5 C5 — `kernscope-pass`: LLVM passes . . . 7
4.6 C6 — `kernscope-profile`: profiler . . . 8

5 Methodology **8**
5.1 Dual-path hardware strategy: CUDA compile-only + HIP live . . . 8
5.2 The four-layer test convention . . . 8
5.3 Honesty guards . . . 9
5.4 Measured machine peaks (gfx1101, RX 7800 XT) . . . 9

6 Evaluation **9**
6.1 End-to-end loop (demo, 2026-08-15, gfx1101) . . . 9
6.2 Verifier corpus (projected) . . . 10
6.3 Pass deltas (projected, compile-only) . . . 10
6.4 Tuning (measured, gfx1101) . . . 10
6.5 Analyzer goldens (projected) . . . 10

7 Limitations **10**

8 Future Work **11**

9 Conclusion **11**

1 Introduction

1.1 Motivation

Writing high-performance GPU kernels is a multi-front activity. A developer must (a) understand where time goes (profiling), (b) capture a failing or hot launch exactly as it happens (record/replay), (c) understand the compiled ISA (static analysis), (d) generate and select among launch-configuration variants (autotuning), (e) guarantee that a variant does not introduce out-of-bounds accesses, races, or barrier divergence (verification), and (f) rewrite the kernel at the IR level (compiler passes). In practice these tasks are served by unrelated tools: `nvprof`/`Nsight Compute` or `rocprofiler` for profiling, CUPTI or rocprofiler-sdk samples for tracing, GPUVerify-class academic verifiers for races, template-based autotuners, and bespoke LLVM passes. Each tool re-derives the same architecture constants (register files, shared-memory sizes, occupancy limiters), each uses a different report format, and none validates the others' outputs.

kernscope attacks this fragmentation with a single repository: six CLIs sharing a library stack (`libs/`), a common report engine with schema version and provenance, and cross-tool edges that make the tools *validate each other*. The profiler's "memory-bound" hint is cross-referenced against the static analyzer's findings on the *captured* kernel's ISA; the tuner gates variants through the verifier before compiling them; the LLVM passes measure their before/after delta with the analyzer's own metrics. This mirrors the loop that hardware-accelerator teams actually run, and it is the report's central claim: that a unified toolchain produces evidence no single tool could vouch for alone.

1.2 Contributions

- **Six integrated CLIs** (`tools/`): `kernscope-analyze` (C1), `kernscope-tune` (C2), `kernscope-verify` (C3), `kernscope-capture` (C4), `kernscope-pass` (C5), and `kernscope-profile` (C6).
- **A shared library stack** (`libs/`): architecture DB with *measured* gfx1101 peaks, PTX/AMDGPU parsers, occupancy and roofline engines, a correctness gate, a timing harness, a report engine, a CUPTI subscription layer, and a runtime wrapper with compile-only + mock backends.
- **A closed tool loop** wired end-to-end by `scripts/demo.sh`, emitting one aggregated cross-tool report (`reports/demo/report.json`).
- **A dual-path hardware strategy** that makes the suite fully testable with zero NVIDIA rentals: every NVIDIA capability has a compile-only implementation, while the HIP equivalents run live on the local gfx1101.
- **A four-layer test convention** and an honesty regime (provenance-labeled reports, owner-gated rentals) asserted by the CI gate.
- **Measured, provenance-labeled evidence**: this report quotes exact numbers from committed JSONL/JSON artifacts, never fabricated values.

1.3 Reading guide

Section 2 positions each tool against prior art (the project's own research briefs). Section 3 describes the system architecture with three TikZ diagrams. Section 4 details the six tools with measured results. Section 5 states the dual-path hardware strategy, the four-layer test convention, and the honesty guards. Section 6 summarizes the evaluation on gfx1101. Sections 7–9 discuss limitations, future work, and conclusions. All measurements are quoted with their exact provenance path.

2 Background and Related Work

The project grounds itself in a set of committed research briefs (`docs/research/*.md`) — one per tool plus `owner-examples.md` and `rdna3-constants.md`. Each brief surveys prior art and cites primary sources; the references in this report are drawn from those briefs and verified where noted in the README.

2.1 Roofline and the occupancy model (foundational)

The performance-modeling foundation is the roofline model of Williams et al. (2009), which bound a kernel's attainable performance by the minimum of its compute ceiling and its memory-bandwidth ceiling, with a ridge point at the arithmetic intensity (FLOPs/byte) where the two cross. kernscope's `libs/roofline` implements exactly this: given a kernel family and a machine peak, it computes arithmetic intensity, ridge point, and bound class. The occupancy model follows the CUDA C++ Programming Guide's definition of occupancy as "the ratio of the number of active warps to the maximum number of active warps supported by the SM" (NVIDIA, 2026b), with limiters computed from device properties (registers, shared memory, threads, warps) exactly as `cudaOccupancyMaxActiveBlocksPerMultiprocessor` does. On the AMD side the guide's constants are mirrored in `rdna3-constants.md` and validated against LLVM's own `GCNSubtarget::computeOccupancy()` machinery. Volkov (2010) motivates low-occupancy-but-high-ILP variants, which the tuner's shared-memory-limited configurations exercise. Nsight Compute's *Speed of Light* methodology (NVIDIA, 2026f) provides the diagnostic vocabulary the profiler adopts (compute-vs-memory percent of peak, occupancy limiter, warp-stall reasoning).

2.2 Static analysis of GPU assembly (C1)

The analyzer brief surveys microbenchmark-driven architecture dissections: Volta (Jia et al., 2018), Turing (Jia et al., 2019), Hopper (Luo et al., 2024), a microbenchmark-driven analytical model across modern architectures (Jarmusch and Chandrasekaran, 2026), and CDNA3 shared-memory/latency exploration (Tee and et al., 2025); the older Wong et al. (2010) dissected Fermi. Simulation frameworks (GPGPU-Sim (Bakhoda et al., 2009), Accel-Sim (Khairy et al., 2020), MGPUSim, gem5) and binary instrumentation (NVBit (Vijaykumar and et al., 2019)) are referenced as alternatives that are far too heavy for a quick static estimate. Ma et al. (2017) grounds the register-limited occupancy sweeps in measured microarchitecture behavior. The analyzer's core (register pressure, occupancy + limiter, coalescing, bank conflicts, warp divergence, barrier usage, tensor-core/MFMA/WMMA detection, spill detection) is a static re-implementation of what these works measure dynamically.

2.3 Autotuning and tensor-program compilation (C2)

The tuner's prior art is the DSL/template generation line: Halide (Ragan-Kelley et al., 2013) (algorithm/schedule separation and autoscheduling), TVM (Chen et al., 2018) and its answer to template-bound search, Ansor (Zheng et al., 2020), and Triton (Tillet et al., 2019) (tile-level IR with an autotuner over tile parameters). The brief additionally cites the Halide autoscheduler variants (`mullapudi2016` analytic, `li2018` RL, `anderson2021` beam search) and Adams et al. (2019) (tree-search scheduling). For attention workloads the memory-bound claim follows FlashAttention (Dao et al., 2022) ("most operations in Transformers are bottlenecked by memory accesses"), with decode-phase variants following FlashDecoding++ (Hong et al., 2024). Vendor tensor-core libraries (NVIDIA, 2026a,c) and CUTLASS (NVIDIA, 2026e) define the bar the tuner's generated matmul kernels are compared against; SCALE-Sim (Samajdar et al., 2019) represents the simulator-based alternative. Kernel Tuner (van Werkhoven, 2024) is the closest standalone autotuner prior art.

2.4 GPU kernel verification (C3)

The verifier brief is the richest. The methodological anchor is GPUVerify (Betts et al., 2012, 2015): SDV semantics and the two-thread reduction that reduces race/divergence checking to sequential program verifica-

tion (with the CAV 2014 engineering retrospective (Deligiannis et al., 2014)). The line of symbolic execution begins with PUG (Li and Gopalakrishnan, 2010) (SMT-based verification of CUDA kernels), continues with SESA (Li et al., 2014) and GKLEE (Li et al., 2012). ESBMC-GPU (Pereira et al., 2017) contributes context-bounded model checking including array bounds and pointer safety. Faial/FaialAA (Liew et al., 2024) brings behavioral types over index expressions; its finding that of 191 DRF kernels **98% need a specific thread-configuration assumption and 27% need user assertions** directly motivated kernscope-verify's launch-configuration ingestion (Exact/UpperBound/Symbolic tiers). Recent work includes HiRace (Jacobson et al., 2024), HGRD (HGRD authors, 2026) (MLIR-level, dominance-based barrier intervals, host-parameter constraints), and the dynamic race detectors GRace (Zheng et al., 2011), GMRace (Zheng et al., 2014), CURD (Peng et al., 2018), BARRACUDA (Eizenberg et al., 2017), ScoRD (Eizenberg et al., 2020), and iGUARD (Kamath and Basu, 2021). LLOV (Afzal et al., 2020) demonstrates static race freedom on LLVM IR; SPMD divergence abstraction (Rosemann et al., 2021) underpins the barrier checks. The engineering template is the Linux eBPF verifier (Linux kernel, 2026): a typed-register + bound-refinement engine with `check\_map\_access()`-style per-check probes, which kernscope-verify mirrors with SCEV/LVI range machinery instead of tnums. SPIR-V validation (Khronos Group, 2026) and MemorySanitizer (Serebryany and Stepanov, 2015) (for the uninitialized-read class) complete the picture.

2.5 Capture / replay (C4)

The capture brief positions the work against Mneme (Parasyris et al., 2023) (kernel record/replay for tuning), the LLNL Record-Remix-Replay line (LLNL, 2026), Kerncap (Ramos and Lowery, 2026) (HSA-level extraction and isolation for HIP/Triton with VA-faithful replay — the closest complete system), Tally (Tally authors, 2024), SIP (SIP authors, 2024), KPerfIR (KPerfIR authors, 2025), and XSched (OSDI 2025, source of `cuda-intercept`). On the tooling side, CUPTI (NVIDIA, 2026d), PyTorch Kineto (PyTorch, 2026), and rocprofiler-sdk (AMD, 2026) define the two vendor interception venues the suite uses. The project explicitly chose CUPTI callback/injection over LD_PRELOAD (research overturned the draft design) for NVIDIA, and rocprofiler-sdk interception for the live HIP path.

2.6 LLVM passes (C5)

The memory-pass brief anchors on Triton (Tillet et al., 2019), IREE's MLIR GPU passes (Google, 2026) (GPUPromoteMatmulOperands, GPUReduceBankConflicts, GPUConvertToCoalescedDMA), MLIR (Lattner et al., 2021, 2019), LLVM (Lattner and Adve, 2004), and the llvm-tutor (banach-space, 2026) wiring model for out-of-tree plugins. GVN's sparse predicated value numbering (Gargi, 2002) underpins the redundant-load elimination pass.

2.7 Profiling (C6)

The profiler brief surveys CUPTI's activity API (NVIDIA, 2026d), Kineto's correlation-id join and Chrome-trace output (PyTorch, 2026), NVBench's L2-flushed cold timing, and Nsight Compute's sections/rules architecture (NVIDIA, 2026f); on the ROCm side it documents rocprofiler-sdk's live interception and the rocprofiler/omniperf/ omnitrace ecosystem. The profiler re-implements the *no-replay, launch-only* tier of Nsight Compute (duration, grid/block, registers, smem, theoretical occupancy + limiter, spill estimate) from Activity records alone, and degrades gracefully when counters are unavailable.

3 System Architecture

3.1 The closed tool loop

The six tools are deliberately arranged as one loop, documented in `docs/architecture.md` and reproduced here:

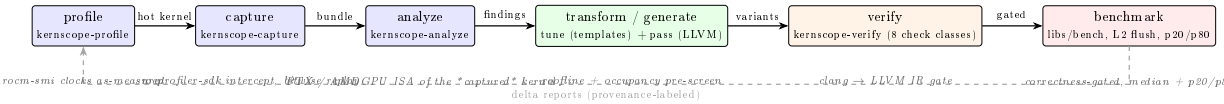


Figure 1: The kernscope closed loop. A kernel enters at *profile* (as a running binary) or at *analyze* (as PTX/AMDGPU ISA) and exits from *benchmark* with provenance-labeled delta reports that feed back into the loop.

The loop is concrete, not decorative: `scripts/demo.sh` runs all five observable stages end-to-end under a 900 s budget (it completes in 153 s on the local machine, `reports/demo/report.json`) and emits one aggregated report. The cross-tool edges that close the loop are: *profile* → *capture* (the profiler ranks hot kernels; capture records the hot one), *capture* → *analyze* (the analyzer runs on the *captured* kernel's disassembled ISA, not a hand-picked example), *capture* → *tune* (`kernscope-tune -from-capture` searches launch configurations over the captured bundle), *tune* → *verify* (`kernscope-tune -verify` gates variants before compiling), *pass* → *analyze* (`kernscope-pass-report -report-pipeline` measures before/after deltas with analyzer metrics), and *profile* ↔ *analyze* (profiler hints carry `analyzer\_cross\_refs` to analyzer finding IDs).

3.2 Library dependency graph

All six tools link the shared stack; nothing depends downward on a tool (`docs/architecture.md`, asserted structurally by CMake):

Notable design decisions visible in the graph: `libs/golden` is a correctness gate (bf16 golden comparisons) rather than a timing helper; `libs/bench` is a timing harness with L2 flush and p20/p80 quantiles; `libs/cupti-sub` is the single CUPTI subscription, shared by capture and profile under a single-tool-scope contract; `libs/runtime` wraps cudart/hiprt with compile-only + mock backends so that compile-gated NVIDIA paths run on AMD hardware; and `tools/pass` shares the analyzer's coalescing classification logic so before/after deltas use identical metrics.

3.3 Dual-path hardware strategy

Because the development machine has only an AMD RX 7800 XT (gfx1101), every NVIDIA capability is given a second, live HIP implementation so the suite is testable with zero rentals:

4 The Six Tools

This section describes each tool's design, implementation notes, and measured results. All numbers are quoted exactly from the committed artifacts listed as provenance.

4.1 C1 — `kernscope-analyze`: static analyzer

Design. `kernscope-analyze` consumes PTX (sm_75–sm_120) or AMDGPU ISA (gfx90a/gfx942/gfx1101) and reports: register pressure, occupancy and its limiting resource, coalescing class per access, shared-memory bank conflicts, warp divergence, barrier usage, instruction mix, tensor-core/MFMA/WMMA usage, spill detection, and latency bottlenecks. The occupancy engine re-implements the vendor formulas with the `libs/arch` constants (per-CC tables from the CUDA guide and `rdna3-constants.md`) without depending on `cuda_occupancy.h`.

Measured results (golden kernel, projected). On the committed `fused\_attention.ptx` fixture (PTX 8.5, sm_86, 64-thread blocks = 2 warps, 67584 B dynamic shared memory), the analyzer computes occupancy 0.375, limiter `registers`, 102 registers/thread, 3 warnings and 3 suggestions (`reports/dashboard.json` analyzer section; golden values in

tests/fixtures/expected_fused_attention.json). In the demo loop the analyzer ran on the *captured* layernorm ISA with 216 ALU ops and produced possibly_divergent_branch (warning) plus info findings (reports/demo/stage-3-analyze.json).

4.2 C2 — kernscope-tune: autotuning kernel generator

Design. Template-based CUDA/HIP kernel generator (families: matmul, bias_gelu, softmax, layernorm, reduction, attention, elementwise). Each variant is *pre-screened* by occupancy + roofline (register/smem budget checks), *gated* by the bf16 golden correctness harness, *timed* by libs/bench (warmup, L2 flush, median + p20/p80), and recorded in a JSON cache keyed by (arch, family, shape-hash). The best variant is reported with its speedup over the naive baseline (best.speedup >= 1.0 is asserted because the baseline is always explored first).

Measured results (gfx1101, live). The committed matmul run (reports/tune/matmul-fnv1a64:e93d312338ce8fd7.jsonl + .tunecache.json, 2026-08-15T14:36Z, provenance.mode=measured) benchmarked 15 variants: the best median is **0.4153 ms** (tile_k=32, 64×64, 4 warps, 1 stage; occupancy 0.375, limiter shared) and the worst is 3.106 ms. The tunecache records 6 pre-screen rejections — e.g. *"register budget exceeded: 512 acc fragment regs/thread (+32 overhead) > 256 max_regs_per_thread on gfx1101"* and *"smem budget exceeded: 66816 bytes > 65536 available"* — and occupancy ranging 0.03125–0.875 across accepted configs. An earlier-generation run is documented in the evidence ledger (.omo/evidence/task-29-gpu-kernel-suite.txt): best 64×64×16, 4 warps, 1 stage at median **0.2027 ms**, roofline position 1 (compute), occupancy 0.875, **speedup 4.2×** vs the naive baseline, 15 of 24 variants measured. The currently committed reports/tune/ artifacts (a later run over a different search space with pre-screening) show best 0.4153 ms; the 0.2027 ms figure is quoted from the ledger, not the current JSONL, and the report treats them as two generations of the same benchmark.

4.3 C3 — kernscope-verify: static verifier

Design. kernscope-verify runs clang over CUDA/HIP source to LLVM IR and applies 8 check classes: (1) out-of-bounds global/shared accesses, (2) shared-memory bounds, (3) barriers (mismatch/divergence), (4) races, (5) illegal pointer arithmetic, (6) divergent barriers, (7) uninitialized reads, (8) invalid address-space casts. Launch configurations are ingested in three tiers (Exact / UpperBound / Symbolic); diagnostics are eBPF-style with a per-function budget; verdicts gate tuner candidates (kernscope-tune -verify). It is a static IR-level verifier, not a formal prover; the soundness scope (in-scope/out-of-scope classes, false-positive/negative classes) is documented in docs/verify-soundness.md.

Measured results (projected corpus). The committed manifest (tests/verify/manifest.json, 34 entries across 19 corpus source files, 8 check classes) yields verdicts: SAFE 18, MAY_OOB 4, PTR_OOB 4, MAY_RACE 2, UNKNOWN_RACE 1, DIVERGENT_BARRIER 1, BARRIER_MISMATCH 1, INVALID_ADDRSPACE_CAST 1, MAY_UNINIT_READ 2 (reports/dashboard.json verify section, provenance.mode=projected). The demo stage-5 report shows the gate in action: one variant was *verifier-gated* out before compilation while 12 others compiled and benchmarked (reports/demo/stage-5-verify-tune.json).

4.4 C4 — kernscope-capture: capture and replay

Design. On AMD, a rocprofiler-sdk intercept (LD_PRELOAD shim) records kernel dispatches with VA-faithful arguments and memory, then replays the bundle standalone with bitwise-equal output validation. On NVIDIA, CUPTI callback/injection (CUPTI_API_ENTER/EXIT, CUDA_INJECTION64_PATH) is primary, with an LD_PRELOAD fallback; the NVIDIA path is compile+unit tested locally and its live behavior is owner-gated. The capture bundle format (v1) is schema-validated (docs/schema/capture-bundle.schema.json).

Measured results (gfx1101, live). The demo captured the hot layernorm kernel (_Z15layernorm_fusedPKtS0_S0_Ptfi.kd, grid 1024×1×1, block 128×1×1) into reports/demo/capture/bundle.v0 (3 blobs) and replayed it with **bitwise PASS**: output

SHA-256 a0027ed9a8ef2e3453aeb194aa64108c1f300a87fbfa292cca5222a725449eec reproduced exactly (reports/demo/report.json stage 2; reports/dashboard.json capture section, replay_bitwise_equal=true).

4.5 C5 — kernscope-pass: LLVM passes

Design. Out-of-tree LLVM-21 plugin (New Pass Manager, PassPluginLibraryInfo, loaded via -fpass-plugin and -load-pass-plugin) implementing: global-to-shared promotion, loop tiling, vectorized load/store rewrite, shared-memory padding, redundant-load elimination, and a coalescing analysis pass. Tests are lit/FileCheck; the before/after delta is measured by kernscope-pass-report -report-pipeline using the *analyzer's own* metrics.

Measured results (projected, compile-only). On the layernorm kernel (reports/passes/layernorm/report.json + llvm-diff.txt + coalesce_before/after.txt, pipeline kernscope-tile,kernscope-promote,kernscope-vectorize,kernscope-pad,kernscope-coalesce): vector width $1 \to 4$, vectorized global loads $0 \to 12$ (pass_reports.txt: "vectorized 48 loads, 0 stores (12 chains)"), widest access 4 B $\to$ 16 B, and global-access count $64 \to 28$ with uncoalesced $64 \to 16$ (delta -48; 12 partial accesses introduced). Register estimate is unchanged at 175 (reports/demo/stage-5-pass-delta.json): improved_metrics = [vectorized_ld_global, vector_width, coalescing], no_transform=false. gemm and softmax were correctly left no-transform; all three kernels compile to PTX with exit 0 after the pipeline (ptx_compile_all_exit0=true). The tile pass declined on layernorm ("no canonical {0,+,1} induction phi"). reports/passes/bench/gfx1101-showcase-bench.jsonl holds the live gfx1101 benchmark of the transformed showcase.

4.6 C6 — kernscope-profile: profiler

Design. Kernel-launch tracing with per-kernel timing, occupancy/register/smem estimates, hot-kernel ranking, and compiler-aware "why slow" diagnostics (limiter decision tree over Activity-only inputs, per docs/profile.md and the profiler brief §6). CUPTI on NVIDIA (compile + unit only); rocprofiler-sdk live on AMD. The diagnostic vocabulary mirrors Nsight Compute's launch-only tier (NVIDIA, 2026f).

Measured results (gfx1101, live). Tracing the layernorm_fused app (reports/demo/stage-1-profile.json): 3 kernels, 19.6 μs wall; the hot kernel _Z15layernorm_fusedPKtS0_S0_Ptfi.kd runs **9200 ns (9.2 μs)** with **24 registers/thread**, occupancy 1 (limiter registers; diagnostic limiter_diag="memory"), 1024 B shared, grid 1024×1×1, block 128×1×1. The profiler emits a memory_bound hint with evidence arithmetic intensity 1.375, bytes-moved estimate 1048576, and ridge point 27.614 (computed from the measured 591 GB/s / 16.32 TF peaks). The same artifact records the peak microbenchmarks (see Section 6).

5 Methodology

5.1 Dual-path hardware strategy: CUDA compile-only + HIP live

The suite pins a reproducible toolchain via devenv (LLVM/MLIR 21.1.8, CUDA 12.6 redistributables for sm_80/sm_90, ROCm 7.2.3; devenv.lock committed and checked by scripts/reproduce-all.sh). Every NVIDIA-runtime capability has a compile-only implementation that is fully exercised on the AMD machine, and a HIP implementation that runs live on gfx1101 (Figure 3 in the architecture diagram, Figure 3). NVIDIA runtime validation is *owner-gated*: only scripts/rental-validate.sh runs NVIDIA code, it is never auto-triggered, and its output lands under reports/rentals/ only when the owner produces it (currently an empty .gitkeep).

5.2 The four-layer test convention

The CI gate (scripts/ci.sh) enforces four layers:

1. **gtest/ctest** – the hardware-labeled suite (`ctest -L "no-gpu|gpu-amd"`) with a non-decreasing count guard (`scripts/ci-baseline.txt`, baseline ≥ 550);
2. **lit/FileCheck** – the `check-kernscope` lit suite for the pass plugin and verifier manifests;
3. **NumPy bf16 golden harness** – `pytest tests/golden`, the reference bf16 golden comparisons that cross-check the C++ correctness gate;
4. **generated-code compile checks** – `scripts/compile_checks.sh`: nvcc sm_80/sm_90 compile-only gates + hipcc gfx1101 compile gates + a live gfx1101 run with `golden pass:true`.

Committed fixtures are capped at 64 MB total; NVIDIA headers are never vendored (the nix store resolves `#include <cuda_occupancy.h>` at build time; ci.sh guards the absence of vendored headers by filename and content).

5.3 Honesty guards

- Every report carries `schema_version: "1.0"` and per-module `provenance.mode` = *measured* (live gfx1101) or *projected* (static IR/PTX artifacts). `tests/report/check_dashboard.py` asserts the honesty flags; projected numbers are never presented as measured.
- The demo pass deltas are explicitly `projected`: the `reports/passes/*` artifacts are `llc` compile-only outputs, and the report notes "the measured numbers are the gfx1101 analog in the libs/db records (mode: measured)".
- `reports/rentals/` is empty; the dashboard machine note says "LOCAL numbers only; rental numbers land under reports/rentals/ only when the owner produces them."
- The gfx1101 peak numbers in `libs/arch` are tagged `peak_source="measured-2026-08-14-gfx1101"` and are asserted (never datasheet numbers) by tests (`arch_test.cpp`, `roofline_test.cpp`, `bench_live_test.cpp`, `check_peaks.py`).

5.4 Measured machine peaks (gfx1101, RX 7800 XT)

`kernscope-bench` (libs/bench, `tool_version kernscope-bench 0.1.0`) measured the machine on 2026-08-14 and committed the raw records to `reports/peaks/gfx1101-bench.jsonl` (device name, `gcnArchName`, rocm-smi sclk/mclk as-measured — clocks were *never* locked, no root). The derived peaks reside in `libs/arch` and `libs/roofline`:

Family	Rep	Median	Derived peak	Provenance
`peak_flops`	FFMA loop	13.51 ms	**16.32 TF**	`gfx1101-bench.jsonl` rows 13/27/41
`peak_bw_read`	512 MiB read	0.907 ms	**591 GB/s**	row 12/26/40; `arch.h` 591.0 GB/s
`peak_bw_d2d`	1 GiB D2D	2.26–2.30 ms	467–475 GB/s	row 11/25/39; `roofline.cpp` 476 GB/s
`peak_lds`	LDS R+W	10.14–10.32 ms	**9.7 TB/s**	row 14/28; `arch.h` 9689.9 GB/s

Table 1: Measured gfx1101 (AMD Radeon RX 7800 XT) machine peaks, 2026-08-14; every row is `provenance.mode=measured`. The fp32 figure is the register-resident FFMA peak (≈15.6–16.4 TF across runs); DRAM read peak 591 GB/s drives the roofline ridge point 27.61 (ratio of the measured peaks 16.32×10^{12} FLOP/s and 591×10^{9} B/s).

6 Evaluation

Hardware. All live numbers are local gfx1101 (AMD Radeon RX 7800 XT, ROCm 7.2.3), measured with as-is clocks. **There are no NVIDIA runtime numbers in this report or in the repository**; PTX/cubin paths are compile-only, and NVIDIA runtime validation is owner-gated via `scripts/rental-validate.sh`.

6.1 End-to-end loop (demo, 2026-08-15, gfx1101)

`scripts/demo.sh` runs the loop in 153 s (budget 900 s) and emits `reports/demo/report.json`. Measured stages:

- **profile** (stage 1): 3 kernels, hot kernel `layernorm\_fused` 9200 ns, 24 regs/thread, occupancy 1, hint `memory\_bound`;
- **capture** (stage 2): bundle.v0 recorded, **bitwise replay PASS**, output SHA-256 reproduced exactly;
- **analyze** (stage 3): analyzer on the *captured* ISA, 216 ALU ops, `possibly\_divergent\_branch` warning;
- **capture-tune** (stage 4): 4 candidates validated over the captured bundle; best block 256 at median **0.011880 ms** (baseline block 128; speedup 1.0017×; the report's own best-block figure is 0.012060 ms, `reports/demo/stage-4-capture-tune.json`);
- **verify-tune** (stage 5): `verify\_gate=true`, 1 variant verifier-gated, 12 compiled; the bias_gelu family's best (block 256, vec 4) runs median **0.17184 ms** vs a 0.20332 ms baseline (speedup 1.183×), occupancy 1 (limiter registers), roofline position 0.0845, memory bound at AI 2.33 (`reports/demo/stage-5-verify-tune.json`).

The cross-tool block of the report proves the loop is closed: the profiler's `memory\_bound` hint carries `analyzer\_cross\_refs` pointing at analyzer finding IDs derived from the captured ISA.

6.2 Verifier corpus (projected)

34 manifest entries, 19 source files, 8 check classes; verdict mix as reported in Section 4 (C3), including the negative-path assurance that a data-dependent index yields `UNKNOWN\_RACE` (not SAFE/MAY_RACE) and that a legal generic-mediated pointer chain is not flagged (`.omo/evidence/task-33/34`).

6.3 Pass deltas (projected, compile-only)

layernorm: vector width 1 → 4, vectorized global loads 0 → 12, uncoalesced accesses 64 → 16 (−48), widest access 4 → 16 B, registers unchanged (175) — `reports/passes/layernorm/*`, `reports/demo/stage-5-pass-delta.json`. gemm/softmax correctly `no-transform`. All before/after PTX compile cleanly (`llc` exit 0).

6.4 Tuning (measured, gfx1101)

matmul: 15 variants measured, best median 0.4153 ms (tile_k=32, 64×64, 4 warps, 1 stage, occupancy 0.375, limiter shared); worst 3.106 ms; 6 pre-screen rejections (register/smem budget); ledger-generation best 0.2027 ms with speedup 4.2× and occupancy 0.875 (`reports/tune/*`, `.omo/evidence/task-29`). flash_attention: 24 compile-gated variants with occupancy 1.0 (limiter registers) for vec1/vec2 tiles and shared-limited 0.25–0.75 for vec4.

6.5 Analyzer goldens (projected)

fused_attention (sm_86, PTX 8.5): occupancy 0.375, limiter `registers`, 102 regs/thread, 67584 B dynamic shared, 3 warnings, 3 suggestions (`reports/dashboard.json`, `tests/fixtures/expected\_fused\_attention.json`).

7 Limitations

- **Single measured platform.** Every live number is gfx1101 (RX 7800 XT). Peaks were measured with as-is clocks (never locked, no root); the same kernel family on CDNA2/CDNA3 or any NVIDIA part is unverified.
- **No NVIDIA runtime evidence.** CUPTI capture/replay and CUPTI profiler tracing are compile+unit tested only; their live behavior on sm_80/sm_90 is unvalidated until the owner runs `rental-validate.sh`. Reports never claim otherwise.
- **Projected pass deltas.** The `reports/passes/*` before/after numbers are static IR/PTX artifacts (`llc` compile-only); the measured gfx1101 analog lives in the libs/db records and the showcase bench. Wall-clock speedups for the vectorized layernorm are therefore not claimed here.
- **Verifier soundness scope.** `kernscope-verify` is a static IR-level verifier, not a formal prover: false-positive and false-negative classes exist and are documented in `docs/verify-soundness.md` (e.g. SCEV-reconstruction failures degrade to `UNKNOWN` verdicts at -O2).
- **Tuning search-space drift.** Two generations of the matmul benchmark (ledger 0.2027 ms vs committed JSONL 0.4153 ms) differ in search space and pre-screening; the report quotes both with provenance rather than averaging.
- **Benchmark noise.** Timing medians with p20/p80 are robust but the machine was shared with development activity; L2 flushing is applied but clock drift is not controlled.

8 Future Work

- **Owner-gated NVIDIA validation.** Run `scripts/rental-validate.sh` on a rented A100/H100-class host; land results under `reports/rentals/` with the same provenance schema, then extend the dashboard with an NVIDIA column.
- **CUDA 13 migration.** The versioned CUPTI record adapter (`tools/profile/.../record\_adapter.h`, `CUPTI\_API\_VERSION` 24) already pins the Kernel9→Kernel10 surface so a header bump is a single-point compile error; exercise it against CUDA 13.
- **Verifier precision.** HGRD-style dominance-based barrier-interval splitting and host-parameter constraint propagation; PTX-level audit layer with spirv-val-style well-formedness gates (HGRD authors, 2026; Khronos Group, 2026).
- **Roofline-driven pass selection.** Wire the profiler's limiter diagnostics to pass selection (vectorize when memory-bound at AI below the ridge, as the layernorm case demonstrates) and to FlashAttention-style IO-aware tiling (Dao et al., 2022).
- **LLM-generated kernel validation.** The verify gate + golden harness is a natural correctness harness for LLM-generated CUDA (cf. CUDAsmith's differential/EMI testing (Zhang et al., 2020)); a KernelBench-style workload sweep is a stated direction in the evidence ledger.
- **Multi-GPU occupancy.** Cluster-scoped shared memory (PTX AS 7) and Hopper/MI300X tile-level models (Luo et al., 2024).

9 Conclusion

kernscope demonstrates that a GPU-kernel tooling suite can be more than the sum of its six CLIs: by sharing a library stack, a report schema, and — critically — the same architectural constants and analysis

metrics, the tools validate each other instead of self-reporting. The closed loop (profile → capture → analyze → transform/generate → verify → benchmark) is run end-to-end in 153 s on a local RX 7800 XT, with the profiler's memory-bound hint cross-referenced against the analyzer's findings on the captured kernel's ISA and the verifier gating every tuner candidate before compilation. The dual-path hardware strategy delivers a suite that is fully testable with zero NVIDIA rentals while keeping NVIDIA claims honest: every number is either measured on gfx1101 (591 GB/s DRAM read, 16.32 TFLOPS fp32, 9.7 TB/s LDS, 0.4153 ms best matmul variant, bitwise-equal capture replay) or explicitly projected from compile-only artifacts. The repository's committed research briefs ground every design decision in prior art from GPUVerify to Kerncap, and this report inherits that grounding. The result is a reproducible, honest, and cross-validated evidence base for GPU kernel optimization — and a template for how toolchains for one domain should be built.

References

Andrew Adams, Krzysztof Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Ben Steinberger, Stefan Johnson, Michaël Gascuel, Kevin Smith, Shahram Izadi, and Frédo Durand. Learning to optimize Halide with tree search and random programs. In *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, volume 38, 2019.

Ayesha Afzal, Ganesh Balakrishnan, and et al. LLOV: A fast static data-race checker for OpenMP programs. *ACM Transactions on Architecture and Code Optimization (TACO)*, 17(4), 2020. doi: 10.1145/3418597.

AMD. ROCm rocprofiler-sdk. https://rocm.docs.amd.com/projects/rocprofiler-sdk/, 2026. HIP_RUNTIME_API + kernel dispatch interception; correlation_id join; replay target of kernscope-capture.

Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. Analyzing CUDA workloads using a detailed GPU simulator. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 163–174, 2009.

banach-space. llvm-tutor: A collection of out-of-tree LLVM passes and tests. https://github.com/banach-space/llvm-tutor, 2026. The standard wiring model for out-of-tree pass plugins with lit tests.

Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. GPUVerify: A verifier for GPU kernels. In *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 113–132, 2012. doi: 10.1145/2384616.2384625.

Adam Betts, Nathan Chong, Alastair F. Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson. The design and implementation of a verification technique for GPU kernels. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 37(3):10:1–10:49, 2015. doi: 10.1145/2743017.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594, 2018. URL https://arxiv.org/abs/1802.04799. arXiv:1802.04799.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS 35)*, 2022. URL https://arxiv.org/abs/2205.14135. arXiv:2205.14135.

Pantazis Deligiannis, Alastair F. Donaldson, Jeroen Ketema, Daniel Liew, and Shaz Qadeer. Engineering a static verification tool for GPU kernels. In *Proceedings of the 26th International Conference on Computer Aided Verification (CAV)*, pages 226–242, 2014.

Alexander Eizenberg, Yuanfeng Peng, Tommaso Pigli, William Mansky, and Joseph Devietti. BARRACUDA: Binary-level analysis of runtime RAces in CUDA programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017. doi: 10.1145/3062341.3062342.

Alexander Eizenberg, Shiliang Hu, and Joseph Devietti. ScoRD: A scoped race detector for GPUs. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020. doi: 10.1109/ISCA45697.2020.00088.

Karthik Gargi. A sparse algorithm for predicated global value numbering. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 45–56, 2002.

Google. IREE: Intermediate representation execution environment (GPUPromoteMatmulOperands, GPUReduceBankConflicts, GPUConvertToCoalescedDMA). https://github.com/iree-org/iree, 2026. MLIR-based production analog for shared-memory promotion and coalescing passes.

HGRD authors. Towards an accurate GPU data race detector. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.02106. arXiv:2604.02106.

Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. FlashDecoding++: Faster large language model inference on gpus. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2311.01282. arXiv:2311.01282.

Tyler Jacobson, Martin Burtscher, and Ganesh Gopalakrishnan. HiRace: A data-race detector for GPU kernels. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2401.04701. arXiv:2401.04701.

Lucas Jarmusch and Sunita Chandrasekaran. Microbenchmark-driven analytical performance modeling across modern GPU architectures. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2605.04178. arXiv:2605.04178.

Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scipioni. Dissecting the NVidia Volta GPU architecture via microbenchmarking. *arXiv preprint*, 2018. URL https://arxiv.org/abs/1804.06826. arXiv:1804.06826.

Zhe Jia, Marco Maggioni, Jeffrey Smith, and Daniele P. Scipioni. Dissecting the NVidia Turing T4 GPU via microbenchmarking. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1903.07486. arXiv:1903.07486.

Anant Kamath and Arkaprava Basu. iGUARD: In-GPU advanced race detection. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021. doi: 10.1145/3477132.3483545.

Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 473–486, 2020.

Khronos Group. SPIR-V specification. https://registry.khronos.org/SPIR-V/, 2026. Universal limits §2.17; spirv-val as the well-formedness gate.

KPerfIR authors. KPerfIR: In-compiler instrumentation and trace replay for GPU kernels. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2505.21661. arXiv:2505.21661.

Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*, pages 75–86, 2004. doi: 10.1109/CGO.2004.1281665.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: A compiler infrastructure for the end of the specialization era. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1903.10267. arXiv:1903.10267.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021. doi: 10.1109/CGO51591.2021.9370308.

Guodong Li and Ganesh Gopalakrishnan. Scalable SMT-based verification of GPU kernel functions. In *Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 187–196, 2010. doi: 10.1145/1882291.1882320.

Guodong Li, Peng Li, Geoffrey Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. GKLEE: Concolic verification and test generation for GPUs. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 215–224, 2012.

Peng Li, Guodong Li, and Ganesh Gopalakrishnan. Practical symbolic race checking of GPU programs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2014. doi: 10.1109/SC.2014.20.

Daniel Liew, Haniel Barbosa, Tiago Cogumbreiro, and et al. Sound and partially-complete static analysis of data-races in GPU programs (FaialAA). In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2024. doi: 10.1145/3689797.

Linux kernel. Linux kernel eBPF verifier. https://github.com/torvalds/linux/tree/master/kernel/bpf/verifier.c, 2026. Tnum + register-bound analysis, check_map_access(), path-sensitive pruning; the engineering template for kernscope-verify.

LLNL. Record-remix-replay (R^3): Llm evolutionary search and bayesian optimization over source, compiler, and launch configuration on a record-replay engine. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.11109. arXiv:2604.11109.

Weile Luo, Ruibo Fan, Zeyu Li, Dayou Zhang, Yan Liang, and Xing Du. Dissecting the NVIDIA Hopper architecture through microbenchmarking and multiple level analysis. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2501.12084. arXiv:2501.12084; IPDPS 2024 version doi:10.1109/IPDPS.2024.00070.

Wenhao Ma, Chunlin Wang, and et al. Understanding the GPU microarchitecture to achieve bare-metal performance. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 31–42, 2017.

NVIDIA. cuBLAS: CUDA basic linear algebra subroutines library. https://docs.nvidia.com/cuda/cublas/, 2026a.

NVIDIA. CUDA C++ programming guide. https://docs.nvidia.com/cuda/cuda-programming-guide/, 2026b. v13.3; occupancy model and launch configuration worked examples (§2.3.7).

NVIDIA. cuDNN: NVIDIA CUDA deep neural network library. https://docs.nvidia.com/deeplearning/cudnn/, 2026c. Tensor-core kernels; implicit-GEMM convolutions referenced by Triton and FlashAttention evaluations.

NVIDIA. CUDA profiling tools interface (CUPTI). https://docs.nvidia.com/cupti/, 2026d. Activity API; Kernel9 vs Kernel10 record kinds; Event/Metric APIs removed in CUDA 13.

NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines and solvers (incl. CuTe layout library). https://github.com/NVIDIA/cutlass, 2026e.

NVIDIA. NVIDIA Nsight compute profiling guide – speed of light (SOL) analysis. https://docs.nvidia.com/nsight-compute/ProfilingGuide/, 2026f. Sections and rules architecture: SpeedOfLight, Occupancy, SchedulerStats, WarpStateStats.

Konstantinos Parasyris, Giorgis Georgakoudis, Enrique Rangel, Ignacio Laguna, and Johannes Doerfert. Scalable tuning of (OpenMP) GPU applications via kernel record and replay. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.

Yuanfeng Peng, Vinod Grover, and Joseph Devietti. CURD: A dynamic CUDA race detector. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 390–403, 2018. doi: 10.1145/3296979.3192368.

Pável Pereira, Higo Albuquerque, Isabela da Silva, Hendrio Marques, Felipe Monteiro, Ricardo Ferreira, and Lucas Cordeiro. SMT-based context-bounded model checking for CUDA programs (ESBMC-GPU). *Concurrency and Computation: Practice and Experience*, 29(22), 2017. doi: 10.1016/j.scico.2017.09.005.

PyTorch. Pytorch Kineto: GPU profiling and tracing library. https://github.com/pytorch/kineto, 2026. CUPTI-based activity tracing with correlation-id join and Chrome trace output.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 519–530, 2013. doi: 10.1145/2491956.2462176.

Cole Ramos and Keith Lowery. Kerncap: Automated kernel extraction and isolation for AMD GPUs. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2605.03208. arXiv:2605.03208 (AMD Research, IntelliKit).

Jannik Rosemann, Simon Moll, and Sebastian Hack. An abstract interpretation for SPMD divergence on reducible control flow graphs. *Proceedings of the ACM on Programming Languages (POPL)*, 5, 2021. doi: 10.1145/3434312.

Ananda Samajdar, Yuhao Zhu, Paul N. Whatmough, Matthew Mattina, and Tushar Krishna. SCALE-Sim: Systolic CNN accelerator simulator. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1811.02883. arXiv:1811.02883.

Konstantin Serebryany and Evgenii Stepanov. MemorySanitizer: Fast detector of uninitialized memory use in C++. In *Proceedings of the 2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 46–55, 2015.

SIP authors. SIP: SASS schedule autotuning via interception. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2403.16863. arXiv:2403.16863.

Tally authors. Tally: Transparent virtualization layer intercepting kernel-launch APIs with PTX rewriting. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.07381. arXiv:2410.07381.

Henry Tee and et al. The MALL is open: Exploring shared caches and latency in AMD CDNA3 GPUs. In *Workshop on Memory-Centric Accelerator and Machine Learning (MEMO), at SC25*, 2025.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

Ben van Werkhoven. Kernel tuner: A search-optimizing GPU kernel auto-tuner (HIP extension). *arXiv preprint*, 2024. URL https://arxiv.org/abs/2407.11488. arXiv:2407.11488.

N. Vijaykumar and et al. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

Vasily Volkov. Better performance at lower occupancy. In *GPU Technology Conference (GTC)*, 2010.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Henry Wong, Mise-Martha Papadopoulou, Maryam Sadooghi-Alvandi, and Andreas Moshovos. Demystifying GPU microarchitecture through microbenchmarking. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 235–246, 2010. doi: 10.1109/ISPASS.2010.5452013.

Zhenyu Zhang, Bo Jiang, W. K. Chan, Yuting Tong, Lei Liu, and Xiaofei Yang. CUDAsmith: A fuzzer for CUDA compilers. In *IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2020.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879, 2020. URL https://arxiv.org/abs/2006.06762. arXiv:2006.06762.

Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. GRace: A low-overhead mechanism for detecting data races in GPU programs. In *Proceedings of the 16th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 135–146, 2011. doi: 10.1145/1941553.1941574.

Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. GMRace: Detecting data races in GPU programs via a low-overhead scheme. *IEEE Transactions on Parallel and Distributed Systems*, 25(1): 104–115, 2014. doi: 10.1109/TPDS.2013.44.

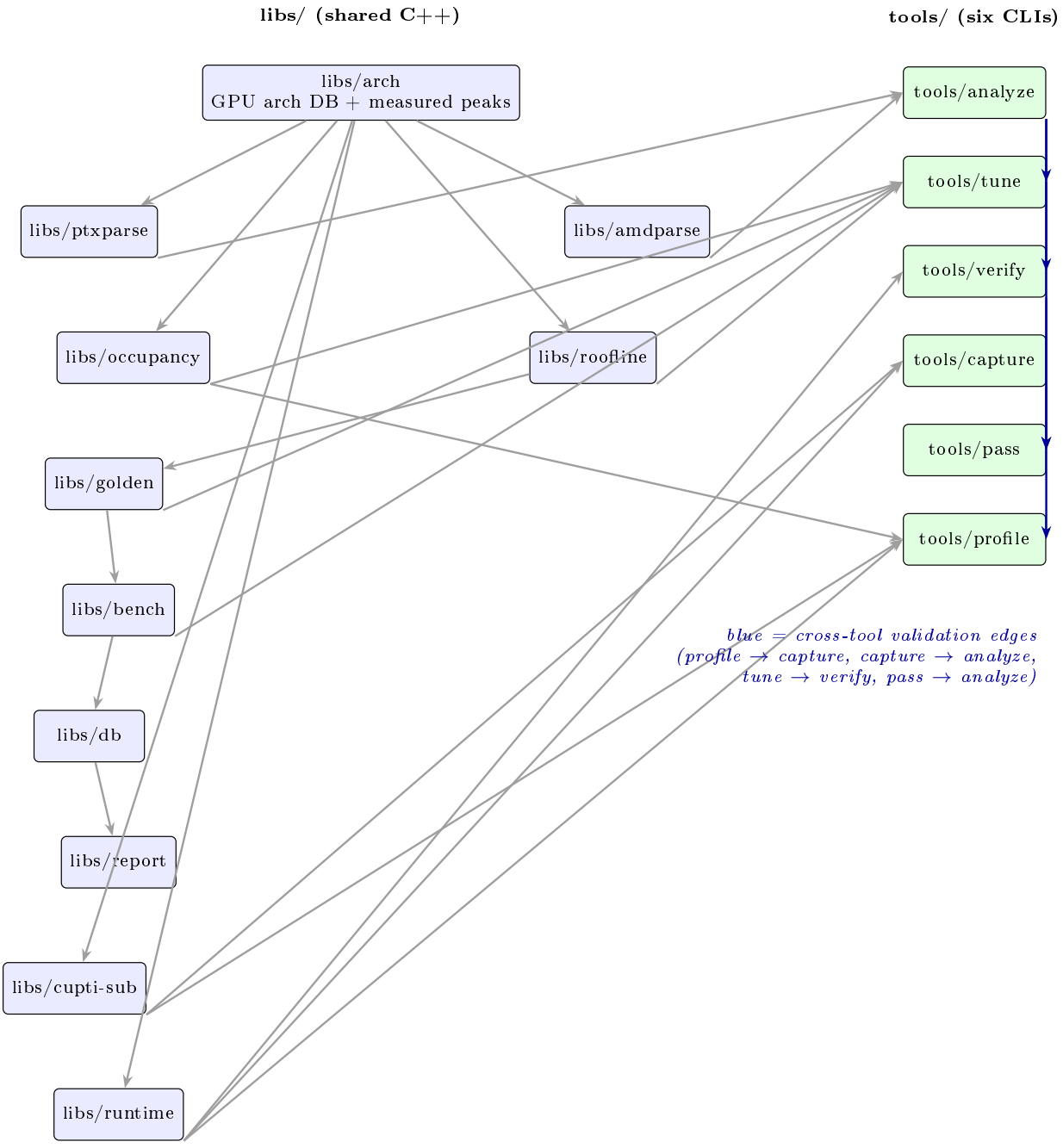


Figure 2: Library dependency graph. `libs/arch` (with its *measured* gfx1101 peaks) roots the stack; parsers feed the analyzer; occupancy/roofline/golden/bench/db feed the tuner; runtime + cupti-sub feed capture and profile. Blue edges are the cross-tool validation edges that close the loop of Figure 1.

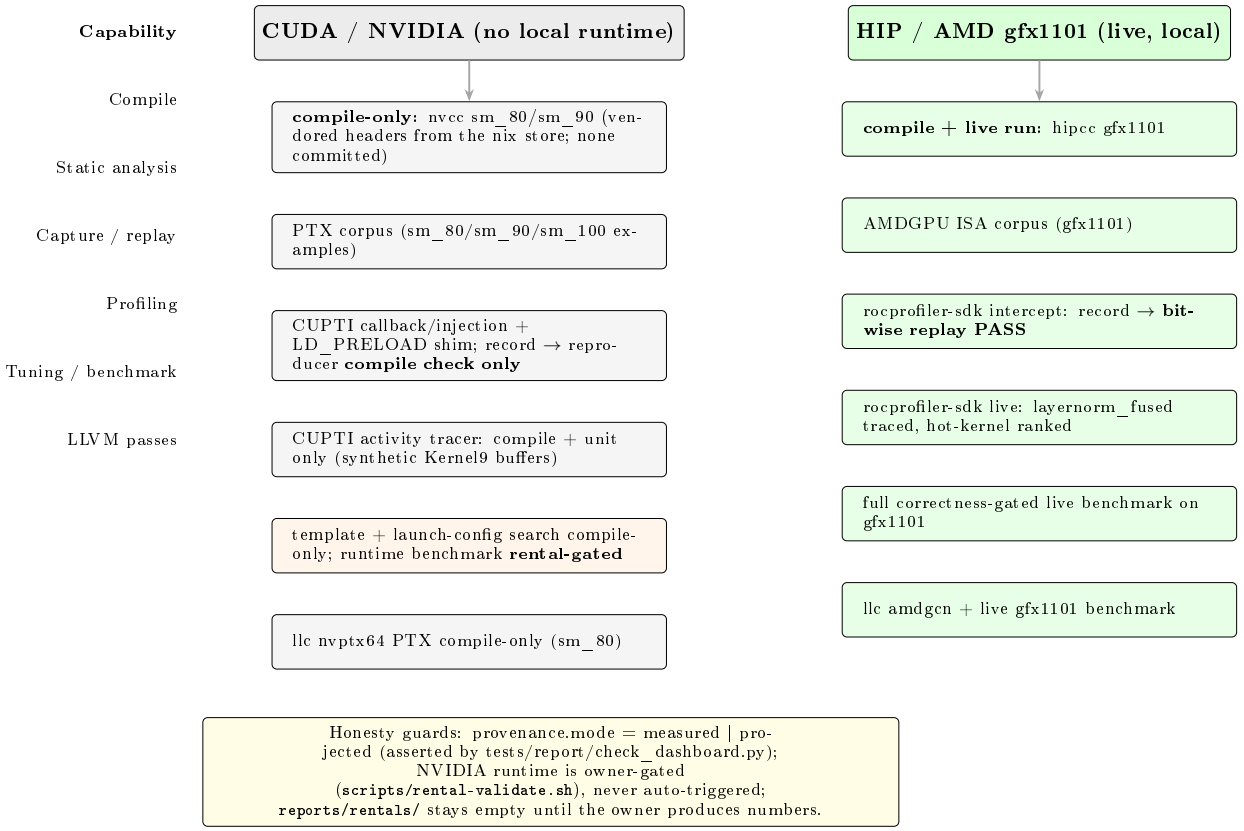


Figure 3: Dual-path hardware strategy. CUDA/NVIDIA capabilities are compile-only or owner-gated rentals; HIP/AMD capabilities run live on gfx1101. The honesty banner (provenance.mode, `reports/rentals/` emptiness, owner-gated `scripts/rental-validate.sh`) is asserted by `tests/report/check\_dashboard.py`.